Changelog

v1.13.0 — 2026-07-06

Game Launch

- **Clarified `session_id` ownership and propagation.** `session_id` is an optional field on `POST /seamless/launch/url`. If the operator sends it, that value is treated as the canonical session identifier and echoed on every wallet callback. If the operator omits it (or sends it blank), the platform generates a canonical `session_id` and applies it consistently across all callbacks — including the initial `player/info` call. Sending `session_id` at launch remains the recommended setup; no operator action is required.
- **Clarified `mode` semantics.** The launch `mode` (`real` / `demo`) is currently informational/reserved: it is recorded on the session and passed to the game client, but does not change wallet behavior. `player/info` is invoked for every launch regardless of `mode`.

Operator integration impact: none — clarification only. Sending `session_id` at launch stays recommended but optional; an omitted or blank `session_id` now receives a single platform-generated value applied consistently across `player/info` and all wallet callbacks.

---

v1.12.0 — 2026-07-01

Bonus API — expired freerounds are now PAID, not reversed

When an `AGGREGATED` freeround grant expires after the player has started playing it, the grant is now **settled** instead of reversed: a single aggregated `FREEBET_WIN` is sent for the total the player accrued across the rounds they played (including `0.00` if they won nothing), exactly as a `PER_ROUND` grant pays as it goes. Previously an aggregated **rollback** was sent and the accrued winnings were not credited — so the same play could pay under per-round settlement but be forfeited under aggregated. Settlement mode is a wire-presentation choice only; the player's payout is now identical in both modes, including on expiry.

Consequences: - The **aggregated rollback is retired** — it has no remaining trigger (expired grants settle via the WIN; a grant cannot be cancelled once the player has started playing it). See `bonus/aggregated-settlement.md` — Aggregated rollback (retired). - The aggregated WIN's `session_id` may be expired (the grant timed out); operators **must not** reject it on session grounds, per the existing rule already documented for the aggregated WIN.

Operator integration impact: no required change. Operators that handle the aggregated `FREEBET_WIN` already settle expired-but-played grants correctly via the existing WIN path. Any aggregated-rollback handler may be kept but will no longer be invoked for freeround grants. If your reconciliation assumes "expired grants are never paid", update that assumption — an expired aggregated grant the player used is now settled with a `FREEBET_WIN`.

v1.11.0 — 2026-07-01

Wallet API — aggregated freeround BET must not be rejected on session

The aggregated freeround BET (`POST /seamless/bet` with `round_id = aggregated:<grantId>`) is sent when the player first uses the grant. Like the aggregated WIN and rollback, it may arrive after the player's game session has ended — the `session_id` may no longer be active (for example, on a late retry following an outage). The Operator **must not** reject the aggregated BET because of an expired or inactive session; identify the grant via `gift_id` + `user_id`. This brings the aggregated BET in line with the rule already documented for the aggregated WIN (`wallet/win.md`) and rollback (`wallet/rollback.md`). Per-round freeround bets are live player actions and are unaffected.

Operator integration impact: required for operators that validate `session_id` on bets. Stop rejecting the aggregated freeround BET (`round_id = aggregated:<grantId>`) on an expired or inactive session — return `result_code: 0` and identify the grant via `gift_id` / `user_id`. Operators that do not session-validate bets are unaffected.

v1.10.0 — 2026-06-29

Bonus API — idempotent create/cancel now report a `duplicate` flag

`POST /grants` and `DELETE /grants/{grantId}` already behaved idempotently — re-issuing an existing `grant_id` returns the original grant, and cancelling an already-`CANCELLED` grant returns it unchanged, both with `result_code: 0`. Those responses now additionally carry two fields (inside the `data` object) so a replay is distinguishable

from a first-time operation:

Field	Meaning
`duplicate`	`true` when the call was an idempotent replay (the grant already existed / was already cancelled); `false` on a first-time operation.
`duplicate_reason`	`"ALREADY_EXISTS"` (duplicate create) or `"ALREADY_CANCELLED"` (cancel of an already-cancelled grant); `null` otherwise.

`duplicate: true` is also returned when two creates of the same `grant_id` race concurrently — exactly one is the fresh issue, the rest report `ALREADY_EXISTS`, and no second grant is created. **Unchanged:** cancelling a non-existent grant is still an error, and cancelling a grant that has already started/completed/expired still returns `1014` / `1015` (a distinct non-cancellable state — not a duplicate). The two fields are part of the response `data` object and are `null` on `GET /grants/{grantId}` and list responses. See **`bonus/grants.md` — Duplicate detection**.

Operator integration impact: optional, additive. If you map idempotent replays to a distinct response (e.g. an "already created / already cancelled" shape) for your own upstream, read `duplicate` / `duplicate_reason` instead of diffing the grant body. Integrations that ignore the new fields are unaffected — `result_code` stays `0` and the grant body is unchanged.

v1.9.0 — 2026-06-07

Bonus API — EXPIRED grants now carry `total_win`

`GET /grants/{grantId}` (and grant list responses) now return `total_win` for `EXPIRED` grants the same way they already do for `COMPLETED` ones — the sum of `CASHED_OUT` bet payouts on the grant, frozen at the moment the grant expired, never updated thereafter.

This closes the gap noted in v1.8.0's status table, where `EXPIRED` was marked `null for now — populates in a subsequent release`. The semantic is identical to `COMPLETED`: a one-shot terminal snapshot of the freeround winnings actually earned under the grant. `CANCELLED` remains `0.00` (cancel is only legal pre-play).

Backfill note: this populates from the moment of release forward. Grants that reached `EXPIRED` before the release ship with `total_win = null` permanently — that is the published contract for historical data. A one-shot reconciliation can populate them if needed; raise a support request.

Operator integration impact: optional. Reconciliation / reporting tooling that already reads `total_win` on `COMPLETED` grants picks up `EXPIRED` grants with the same shape — no schema change, no new field, the column just stops being null for the `EXPIRED` subset.

v1.8.0 — 2026-06-07

Bonus API — `total_win` on grant responses (terminal snapshot)

`GET /grants/{grantId}` (and grant list responses) now include a new field, `total_win`, carrying the player's freeround winnings on a grant. The field is **lifecycle-aware**: null while the grant is still being played, populated with the final value once the grant reaches a terminal status.

Semantics by status:

Status	`total_win`
`PENDING`, `ACTIVE`, `IN_PROGRESS`, `EXHAUSTED`	`null`
`COMPLETED`	terminal snapshot (sum of `CASHED_OUT` bet payouts, frozen at the COMPLETED transition)
`CANCELLED`	`0.00` (cancel is only legal pre-play, so no winnings are possible)
`EXPIRED`	`null` in v1.8.0 — populates in v1.9.0 (see v1.9.0 entry above)

The snapshot is written exactly once at the terminal transition and never updated thereafter (it is not a running counter). For live winnings during play, derive from the `FREEBET_WIN` wires you receive — that remains the authoritative money source during a grant's lifetime. `AGGREGATED` grants emit only one grant-level WIN at terminal (per `bonus/aggregated-settlement.md`), so for AGGREGATED grants the `total_win` field is the natural way to learn the final amount.

Operator integration impact: optional. If your support / reconciliation tooling already polls `GET /grants/{grantId}` for terminal state, you now have the winnings total in the same response — no separate wire-aggregation step needed for `COMPLETED` grants. If

you ignore the field, nothing changes for you.

v1.7.0 — 2026-06-06

Bonus API — AGGREGATED freeround settlement available

A new `settlement_mode` value, `AGGREGATED`, is now available on freeround offers. The operator-visible outcome per AGGREGATED grant is one of three shapes — `BET + WIN`, `BET + ROLLBACK`, or no wire activity at all — instead of the per-round `BET/WIN` pairs that `PER_ROUND` produces. See **`bonus/aggregated-settlement.md`** for the full wire shape, lifecycle, and failure-handling rules.

Phased rollout — initially enabled on `king_move` only. Other freeround-capable games (slots, plinko, mines) stay `PER_ROUND`-only until the platform widens their game-bonus configuration. Query `GET /games/{gameId}/bonus-support` per game to see whether `AGGREGATED` is listed in `settlement_modes`.

What's new on the wire (only fires for AGGREGATED grants):

- **Aggregated BET** — `POST /seamless/bet` sent once on first successful use of an AGGREGATED grant. `bet_type=FREEBET`, `amount=0.00`, `gift_id=<grantId>`, `transaction_id=<grantId>-aggregated-bet`, `round_id=aggregated:<grantId>`. No per-round bet wires are sent for that grant.
- **Aggregated WIN** — `POST /seamless/win` sent at the grant's terminal `COMPLETED`. `win_type=FREEBET_WIN`, `amount=sum of per-round netted wins`, `parent_transaction_id=<grantId>-aggregated-bet`. Same `round_id` sentinel.
- **Aggregated ROLLBACK** — `POST /seamless/rollback` sent if the grant reaches `EXPIRED` / `CANCELLED` after the BET wire was acknowledged. `transaction_id=<grantId>-aggregated-rollback`, `parent_transaction_id=<grantId>-aggregated-bet`.

Pair invariant — the operator's ledger never sees a half pair. If GameHub never delivers the aggregated BET (terminal reject by operator, or retry budget exhausted), no WIN or ROLLBACK is sent for that grant. See aggregated-settlement.md — Failure handling.

Other changes:

- `bet_mode=BET_FROM_WIN` combined with `settlement_mode=AGGREGATED` is now supported (was previously rejected at offer creation with `OFFER_BET_MODE_INCOMPATIBLE`). Per-round `BET_FROM_WIN` netting applies as usual; the aggregated WIN is the sum of the per-round netted amounts.

Operator integration impact: required if you want to record the grant-level wire activity in your ledger. If your wallet already accepts `FREEBET` / `FREEBET_WIN` per the per-round freeround spec, the aggregated calls arrive without protocol changes — the wire shape is the same; only the `transaction_id` / `round_id` / `gift_id` semantics scope to the grant rather than the round. Existing `transaction_id` idempotency rules apply unchanged. If you do not yet plan to record grant-level wires, you can ignore them; the wire shape is structured so that echoing `result_code: 0` passively produces consistent ledger entries.

v1.6.0 — 2026-05-22

Wallet API — freeround flow documentation

Three operator-facing docs updated with explicit "Freeround bets / wins / rollbacks" sections. No wire-protocol change; this is a documentation pin of fields that were already being sent but weren't called out as a coherent flow.

What's documented now:

- **`wallet/bet.md` — Freeround bets**: the three correlated fields (`bet_type=FREEBET`, `amount=0.00`, `gift_id=<grantId>`) plus the freeround-specific `attributes.configuredBet`. The configured-bet attribute carries the **actual amount the player chose** for the round (in the bet's currency, decimal-formatted to currency precision). For fixed-amount offers this equals the fixed value; for **range** offers — where the campaign configures a min/max and the player picks within it — `configuredBet` is the player's choice, distinct from `amount` (which is always `0.00` on freerounds since the wallet isn't debited).
- **`wallet/win.md` — Freeround wins**: the three correlated fields on the settlement side (`win_type=FREEBET_WIN`, `gift_id=<grantId>`, `parent_transaction_id` linking to the bet) and an explicit cross-reference to the existing `BET_FROM_WIN` netting rule.
- **`wallet/rollback.md` — Freeround rollbacks**: `gift_id` echoed from the placement; `parent_transaction_id` points back at the original `FREEBET`. Rollback requests carry no `amount` field at all and `attributes` is empty — the stake (`configuredBet`) is recovered from the parent bet record via `parent_transaction_id`. Balance unchanged because the original `FREEBET` was sent with `amount: 0.00`.

Why this matters for integration:

- The `configuredBet` attribute lets operators record the player's actual freeround stake in their internal RTP / KPI / reporting systems **without ever debiting the wallet**. The wallet-level `amount` represents balance impact (always `0.00`); `configuredBet`

represents game-side stake. These are deliberately separate.

- The `gift_id` field consistently appears on bet, win, and rollback for the same freeround round — operators can use it as a join key to reconstruct the per-grant lifecycle.
- Responsible-gaming wagering limits should be compared against `configuredBet`, not `amount`, on freerounds.

Operator integration impact: none required if your wallet already accepts `FREEBET` / `FREEBET_WIN` and stores `gift_id`. If you previously ignored `attributes.configuredBet`, consider capturing it for RTP / activity reporting.

v1.5.0 — 2026-05-21

Wallet API — GameHub retry policy now documented

The behaviour of GameHub when calling your wallet endpoints (`/seamless/bet`, `/seamless/win`, `/seamless/rollback`) is now spelled out in `error-handling.md` — *GameHub Retry Policy*. No wire-protocol changes; this is a documentation pin of behaviour that already existed but was previously implicit.

Key points integrators should verify:

- Returning **HTTP 200 +** `result_code != 0` is the only way to terminally reject a bet on the first attempt. **HTTP 4xx will be retried** through the full backoff ladder before GameHub gives up.
- A non-2xx response is retried up to **16 times over roughly 45 hours** before GameHub stops attempting that `transaction_id`.
- Your wallet MUST be idempotent on `transaction_id` — GameHub reuses the same `transaction_id` across retries.

Why now: a recent operator-side validator bug returned HTTP 422 for a malformed `amount` field. GameHub previously surfaced this as a transport error and retried indefinitely; the retry budget is now bounded at 16 attempts. No change to operator-side behaviour is required.

v1.4.0 — 2026-05-06

Bonus API (Free Rounds) — `BET_FROM_WIN` bet mode now supported

`POST /offers` now accepts `bet_mode = BET_FROM_WIN` (previously rejected as "not yet

supported"). Both modes are also reported by the discovery endpoint `GET /games/{id}/bonus-support` under `bet_modes`.

Semantics:

Mode	Wallet credit at settlement
`ZERO_BET` (default, unchanged)	`nominalWin`
`BET_FROM_WIN` (new)	`max(0, nominalWin - stake)`

The wire-protocol fields are identical for both modes (`bet_type=FREEBET` at placement, `win_type=FREEBET_WIN` at settlement). Only the `amount` on the win command differs. Zero-net BET_FROM_WIN settlements (e.g. cashout exactly at unit multiplier) still send `FREEBET_WIN, amount=0` — operators must acknowledge as a normal successful settlement, not skip it.

Constraint *(superseded by v1.7.0 — BET_FROM_WIN with AGGREGATED is now supported)*: BET_FROM_WIN is supported only with `settlement_mode = PER_ROUND`. Combining BET_FROM_WIN with AGGREGATED settlement is rejected at offer creation with code `OFFER_BET_MODE_INCOMPATIBLE` — the netting semantics across aggregated rounds are intentionally not yet defined.

Operator integration impact: see `wallet/win.md` for the FREEBET_WIN amount rule. Operators **must not** apply additional deduction; the platform sends the already-netted amount for BET_FROM_WIN.

v1.3.0 — 2026-05-01

Bonus API (Free Rounds) — new `EXHAUSTED` grant status

A new freeround grant status value, `EXHAUSTED`, can now appear in `GET /grants/{id}` and grant-history responses. The lifecycle is:

```
PENDING → ACTIVE → IN_PROGRESS → EXHAUSTED → COMPLETED  (terminal)
                                      ↓
                                (revertible to IN_PROGRESS on cancel
                                 of the bet that exhausted the balance)
{ACTIVE | IN_PROGRESS | EXHAUSTED} → EXPIRED | CANCELLED  (terminal)
```

What it means: `EXHAUSTED` represents "the freeround balance is no longer enough to fund another bet, but the grant lifecycle is still open" — the player's last freeround bet is

in flight (cashable / cancellable). Once that bet settles the grant transitions to `COMPLETED` (truly terminal); if it gets cancelled during the betting window, the grant reverts to `IN_PROGRESS` with balance restored.

Operator action required: where you previously checked `status == 'COMPLETED'` to mean "freeround is done", continue using `COMPLETED` — it now reliably means truly terminal. Where you check "is this grant currently active for this player", include `EXHAUSTED` alongside `ACTIVE` and `IN_PROGRESS`.

Bonus API (Free Rounds) — per-grant amount override on issue

`POST /grants` and `POST /grants/batch` accept two new optional fields letting operators award **different amounts to different players from the same offer template** without creating a new offer per amount:

- `rounds` (Integer) — only valid for fixed-bet offers; resolves to `rounds × offer.bet`.
- `free_round_balance` (Number) — only valid for bet-range offers; used directly.

The two fields are mutually exclusive; supplying both, or supplying the wrong one for the offer's mode, is rejected with `1028 GRANT_AMOUNT_FIELD_MISMATCH`. A zero/negative override is rejected with `1027 GRANT_INVALID_AMOUNT`. A resolved balance below the offer's minimum playable bet is rejected with `1029 GRANT_BALANCE_BELOW_MIN_BET`.

Existing integrations are unaffected. Operators that omit both fields keep getting the offer's default `free_round_balance` exactly as before — no request body shape changes, no behavior changes.

When an override is supplied, the grant response and `GET /grants/{id}` echo the operator's input back as `awarded_rounds` and `awarded_free_round_balance` (read from persisted columns, never recomputed). Both echo fields are `null` on grants issued without an override.

Idempotency unchanged: retried `POST /grants` with the same `grant_id` returns the original grant, including the originally-resolved balance and echoed override fields. First call wins.

---

v1.2.3 — 2026-04-19

Bonus API (Free Rounds)

- **BREAKING: Removed `game_type` field from offer create requests.** `POST`

`/offers/fixed-bet` and `POST /offers/bet-range` no longer accept `game_type`. The server now derives it from `game_id` by looking up the game's bonus support configuration (`GET /games/{gameId}/bonus-support`). If the operator sends a `game_id` that does not support free rounds, the request is rejected with error code `1021` (`OFFER_GAME_NOT_SUPPORTED`). `game_type` is still present in offer *response* bodies for convenience. Operator migration: drop `game_type` from request bodies.

- **BREAKING: Removed `use_period_hours` field from offers.** The offer-level "hours to use after activation" hint has been removed from both `POST /offers/fixed-bet` and `POST /offers/bet-range`, and from the offer response shape. Grant expiry is set explicitly at issue time via the `expiry_date` field on `POST /grants` — this was already the authoritative value and the offer hint was redundant. Operator migration: drop `use_period_hours`; set `expiry_date` on grant creation instead.
- **New validation errors.** Offer creation now validates the submitted bet against the game's bonus support config and rejects with explicit error codes:
  - `1021` (`OFFER_GAME_NOT_SUPPORTED`) — game doesn't support free rounds.
  - `1022` (`OFFER_BET_OUT_OF_RANGE`) — bet (or bet range) falls outside the game's allowed min/max.
  - `1023` (`OFFER_SETTLEMENT_MODE_UNSUPPORTED`) — requested `settlement_mode` is not in the game's supported set.
- **Grant `expiry_date` aligned with campaign window.** Grants issued under a campaign can now omit `expiry_date` — it inherits `campaign.end_date`. When provided, it must fall within the campaign window (inclusive of `campaign.end_date`); a later expiry is rejected with `1024` (`GRANT_EXPIRY_AFTER_CAMPAIGN_END`). Standalone grants (offer has no campaign) now **require** an explicit `expiry_date`; a missing value is rejected with `1025` (`GRANT_EXPIRY_REQUIRED`). The resolved value is snapshotted at issue time, so later campaign window changes do not retroactively affect issued grants.

Wallet Protocol

- **"One win per bet" rule softened to SHOULD.** The rule that the Operator rejects a win whose `parent_transaction_id` already has a recorded win (with a different `transaction_id`) is now a **SHOULD** rather than MUST. The aggregator already derives Win `transaction_id` deterministically from the originating bet (`{bet_transaction_id_stem}-win`), so retries always reuse the same value and are caught by the standard `transaction_id` idempotency check that every Operator already implements. The parent-reference check remains documented as a **nice-to-have defense-in-depth** safety net — implementing it protects the Operator's ledger against any caller that regenerates txids on retry. No action required from Operators

already implementing the previous MUST — this change only downgrades the obligation level.

---

v1.2.2 — 2026-04-20

Wallet Protocol (Free Rounds)

- **Protocol correction: freeround bets now carry `bet_type: FREEBET`.** Previously the Bet API sent `bet_type: BET` for all bets, including those funded by a freeround grant. The wallet documentation (`docs/wallet/bet.md`) has always specified `FREEBET` for free-bet bonus bets — this release aligns the server with the spec. The `reason: FREEROUND`, `grantId`, and `configuredBet` attributes are unchanged.
- **Protocol correction: freeround payouts now carry `win_type: FREEBET_WIN`.** Matching the above, the Win API sends `FREEBET_WIN` (per `docs/wallet/win.md`) when crediting a winning freeround bet. `WIN` is still used for wallet-funded bets; `CLOSE_ROUND` is preserved for round-finalisation signals (not tied to a bet).
- **Operator integrators**: if your wallet implementation validated `bet_type` / `win_type` against an allowlist, ensure `FREEBET` and `FREEBET_WIN` are accepted. Both values are already in the public protocol documentation — this is a bug fix against the spec, not a new feature.

Bonus API (Free Rounds)

- **Historical `total_win` restored on grant history responses.** `total_win` is back on the `GetFreeroundHistory` response (and the Hermes SFS `GetFreeroundHistoryOk` message). Unlike the removed v1.2.1 counter, this value is derived on demand by summing `CASHED_OUT` bet payouts tagged with the grant's `grant_id` — not stored as a mutable field on the grant. Lost / cancelled / rolled-back bets contribute zero.

---

v1.2.1 — 2026-04-20

Bonus API (Free Rounds)

- **BREAKING: Removed `max_win_amount` field from offers.** The aggregate per-grant win cap has been removed from the Bonus API. The field is no longer accepted in `POST /offers/fixed-bet` or `POST /offers/bet-range` requests and will not appear in offer responses. The game-level max win cap (configured at the game

level, applied to every cashout) remains in effect as the per-bet payout ceiling. For per-grant cost control, operators should size offers against the game max win cap: `free_round_balance` (or `rounds x bet`) bounds promotional stake; the game-level max win cap bounds each payout.

- **BREAKING: Removed `total_win` field from grant responses.** The field was unused by the enforcement logic (which has also been removed) and reported as `0` in all responses. It no longer appears in `GET /grants/{grantId}` or grant list responses.
- **Grant completion simplified.** `COMPLETED` status now fires only when the free round balance drops below the minimum bet. The previous "win cap reached" completion reason has been removed. `CANCELLED` and `EXPIRED` remain as separate terminal states.

---

v1.2.0 — 2026-04-01

Bonus API (Free Rounds)

- **New section: Bonus API** — Added complete documentation for the free round award system, accessible at `bonus.playcore.live/v1`. Operators can create campaigns, configure offers, and issue free round grants to players.
- **Discovery endpoint** — `GET /games/{gameId}/bonus-support` returns game-specific bonus configuration including supported offer modes, bet options, and game-type-specific settings.
- **Two offer modes** — `fixed_bet` (operator sets a fixed per-round bet) and `bet_range` (player chooses bet within operator-configured bounds).
- **Campaign tracking** — Optional campaign grouping for GGR reconciliation between provider and operator. Provider creates a campaign, shares the ID with operator, grants issued under it are tracked separately for reporting.
- **Grant lifecycle** — Full lifecycle management: issue, activate, track play progress, cancel, expire. All creation endpoints are idempotent.
- **Authentication** — All Bonus API requests require `X-API-Key` and `X-Operator-Id` headers.

---

v1.1.1 — 2026-03-23

Game Launch

- `exit_url` **UI behavior** — Documented that providing `exit_url` renders a close button in the game header, allowing the player to exit during gameplay. When omitted, no exit button is shown.
- `error_url` **UI behavior** — Documented that providing `error_url` renders a close button on the error overlay, redirecting the player on critical errors. When omitted, the game attempts to close the tab/window.
- **URL Parameter Behavior section** — Added quick-reference table summarizing UI effects of `exit_url` and `error_url`, with a recommendation to always provide both.
- **Extra Parameters section** — Documented supported `extra_params` keys: `sound`, `music`, and `animation` (all boolean, default `true`). These override game client defaults at launch; players can still toggle them at runtime via the in-game settings panel.

---

v1.1.0 — 2026-03-13

New Features

- **Responsible gaming limit codes** — Added error codes `1013` (loss limit), `1014` (wager limit), `1015` (time limit), `1016` (general limit) for bet rejection when player exceeds responsible gaming limits.
- **Insufficient balance error code** — Added `1012` for bet rejection when player's balance is insufficient.
- **Bet not found error code** — Added `1011` for win rejection when `parent_transaction_id` does not exist or the bet is already settled by a previous win.
- **Bet already settled error code** — Added `1017` for rollback rejection when the bet already has a corresponding win or the round is closed.
- **Multi-bet rounds** — Documented support for multiple bets within a single round (e.g. crash games). Each bet receives its own win; `round_finished=1` is sent with the final win.
- **Amount precision for signatures** — Defined currency precision scale (USD, EUR, GEL = 2 decimal places) for X-Sign calculation. The JSON body may contain any valid numeric format; the operator must normalize to the precision scale when building the signing string. Other currencies provided during integration.
- **Timeout guidance** — 1 second for crash games, 5 seconds for all other game types. Documented aggregator behavior on timeout (rollback for bet, retry for

win/rollback).

- **Capacity note** — Aggregator does not apply rate limiting; operator must provision for their player load.

Error Handling

- **HTTP status model** — Clarified two-layer error model: HTTP 200 with `result_code` for all business logic (including auth errors); HTTP 400/403/5xx for infrastructure only, with no response body.
- **Auth errors via HTTP 200** — `1002` (invalid credentials) and `1007` (invalid token) are returned as HTTP 200 with `result_code`, not HTTP 401.
- **No response body on infra errors** — HTTP 400, 403, and 5xx return no structured body.

Wallet API

- **Win rules** — Operator must reject win with `1008` if round is closed; must reject with `1011` if bet not found or bet already has a corresponding win. One win per bet enforced.
- **Win/Rollback session liveness** — Operator must not reject win or rollback due to expired session. Session may no longer be active at settlement time.
- **Rollback error codes** — Rollback now returns explicit error codes: `1011` (bet not found) when `parent_transaction_id` does not exist, `1017` (bet already settled) when the bet already has a win or the round is closed. Success (`result_code: 0`) only when the bet is actually refunded.
- **Balance/currency optional on error** — `balance` and `currency` fields are required only on success (`result_code: 0`); optional on error responses.
- **Bet rules** — Added rules for insufficient balance (`1012`), closed round (`1008`), responsible gaming limits (`1013` – `1016`), and idempotency.

Security

- **TLS 1.2 minimum** — All API communication requires HTTPS with TLS 1.2+. TLS 1.3 recommended.
- **Replay protection** — Documented 3-step validation order (timestamp → nonce → signature). Nonces stored with 30-second TTL matching the timestamp drift window.
- **Amount formatting in signatures** — Added detailed guidance and examples for

amount normalization in X-Sign calculation. The JSON body may contain `1`, `1.0`, or `1.00`, but the signing string must always use the normalized format (e.g. `amount=1.00` for USD). This is the most common integration bug.

OpenAPI

- **Flexible enums** — `bet_type` and `win_type` no longer use strict `enum`. New values may be introduced; operators notified via changelog.
- **Rollback response schema** — Separate `RollbackResponse` schema with error code support (`1011`, `1017`).
- **Consistent naming** — All responses use `result_code` (removed legacy `result` field).

---

v1.0.0 — 2025-10-01

Initial version.